

Appendix B: Practice (Detailed Hinglish Explanation)

Ye appendix theory padhane ke liye nahi hai. Iska main purpose hai **Scope, Closure aur Modules ki understanding ko test aur strengthen karna**.

Kyle Simpson basically keh rahe hain:

"Agar tumne book samajh li hai, to ab real coding exercises karke dekho ki concepts kitne clear hain."

Important baat:

- In exercises ka koi ek "perfect answer" nahi hai.
 - Same problem ko solve karne ke multiple valid approaches ho sakte hain.
 - Focus syntax ya style par nahi hai.
 - Focus hai **scope chain, closure memory aur module pattern** ko practically use karna.
-

1. Buckets of Marbles

Ye exercise directly Chapter 2 aur 3 ke Scope concepts ko test karti hai.

Goal

Aisa program likho jisme:

☒ Kam se kam 6 scopes ho

Including:

- Global Scope
- Function Scope
- Block Scope

Har scope ko alag color assume kar sakte ho.

Example:

```
1 // RED
2 var x = 10;
3
4 function foo() {
5     // BLUE
6 }
```

☒ Har scope me koi identifier ho

Identifier matlab:

```
1 var name;  
2 let age;  
3 function foo(){}  

```

Ye sab identifiers hain.

Har scope khaali nahi hona chahiye.

☒ At least 2 Function Scopes

Example:

```
1 function foo() {  
2  
3 }  
4  
5 function bar() {  
6  
7 }  

```

☒ At least 2 Block Scopes

Example:

```
1 if (true) {  
2     let x = 10;  
3 }  
4  
5 for (let i=0; i<10; i++) {  
6  
7 }  

```

if aur for dono block scopes create karte hain.

☒ Shadowing hona chahiye

Chapter 3 ka important concept.

Example:

```
1 var student = "Kyle";  

```

```
2
3  function foo() {
4      var student = "Sarah";
5  }
```

Yahan inner student

outer student ko shadow kar raha hai.

☒ Scope chain me 2 levels upar wala variable access karo

Example:

```
1  var color = "red";
2
3  function outer() {
4
5      function inner() {
6
7          function deepest() {
8              console.log(color);
9          }
10
11      }
12
13  }
```

Yahan:

```
1  color
```

global scope me hai.

Aur deepest() usse 2 levels upar se access kar raha hai.

Ye lexical scope chain ka proof hai.

Suggested Solution Understanding

Book ne prime-number finder use kiya hai.

Global Scope (RED)

```
1 const howMany = 100;
```

Ye global variable hai.

Function Scope (BLUE)

```
1 function findPrimes(howMany) {
```

Is function ke andar:

```
1 var sieve  
2 var max
```

available hain.

Block Scope (GREEN)

```
1 for (let i=2; ...)
```

i

sirf loop block me exist karta hai.

Block Scope (ORANGE)

```
1 if (sieve[i]) {
```

Yahan

```
1 let j
```

exist karta hai.

Block Scope (PURPLE)

```
1 for (let k = j; ...)
```

Yahan

```
1 k
```

exist karta hai.

Callback Scope (PINK)

```
1 .map(function getPrime({}))
```

Function scope create hua.

Callback Scope (YELLOW)

```
1 .filter(function onlyPrimes({}))
```

Ek aur function scope create hua.

Total:

1. RED
2. BLUE
3. GREEN
4. ORANGE
5. PURPLE
6. PINK
7. YELLOW

Already 7 scopes.

Exercise satisfy ho gaya.

2. Closure (Part 1)

Yeh exercise closure ka sabse practical use sikhati hai:

Caching / Memoization

Problem

Suppose:

```
1 isPrime(4327);
```

Call kiya.

Function 100+ checks karega.

Phir dobara:

```
1 isPrime(4327);
```

Fir wahi saare checks.

Fir:

```
1 isPrime(4327);
```

Fir same checks.

Waste hai.

Better Idea

Pehli baar calculate karo.

Result memory me store kar do.

Second time direct memory se return.

Yehi caching hai.

Before

```
1 isPrime(4327);
```

Compute.

```
1 isPrime(4327);
```

Again compute.

After

```
1 isPrime(4327);
```

Compute.

Store:

```
1 {  
2   4327: true  
3 }
```

Fir:

```
1 isPrime(4327);
```

Direct cache se.

No computation.

Closure ka role

Outer scope:

```
1 var primes = {};
```

Returned function:

```
1 return function isPrime(v) {  
2  
3 }
```

Outer variable ko access kar rahi hai.

Ye closure hai.

Visualize:

```
1 (function(){  
2  
3     var primes = {};  
4  
5     return function(){  
6  
7         primes[11] = true;  
8  
9     }  
10  
11 })();
```

Function finish ho gaya.

Lekin:

```
1 primes
```

destroy nahi hua.

Closure usse alive rakhta hai.

factorize() me bhi cache

Same concept.

```
1 var factors = {};
```

Store:

```
1 {  
2   12: [3,2,2]  
3 }
```

Next time:

```
1 factorize(12)
```

direct result.

Memory vs Speed

Author important warning deta hai.

Gain

Fast execution.

Cost

Extra memory.

Example:

```
1 cache = {  
2   1: ...  
3   2: ...  
4   3: ...  
5   ...  
6  1000000: ...  
7 }
```

Huge memory consumption.

Closure free nahi hai.

Tradeoff:

```
1 Memory ↑  
2 Speed ↑
```

Ya

- | | |
|---|----------|
| 1 | Memory ↓ |
| 2 | Speed ↓ |

Situation par depend karta hai.

LRU Cache

Author briefly mention karta hai.

LRU = Least Recently Used

Idea:

Cache full ho gaya.

Toh jo item longest time se use nahi hua usko remove kar do.

Jaise browser history.

Recent pages retain.

Purane pages remove.

3. Closure (Part 2)

Toggle Function

Ye closure ka bahut classic interview-style example hai.

Goal:

1	<code>var onOff = toggle("on","off");</code>
---	--

Calls:

1	<code>onOff();</code>
---	-----------------------

Output:

1	on
---	----

Again:

1	<code>onOff();</code>
---	-----------------------

Output:

```
1 off
```

Again:

```
1 onOff();
```

Output:

```
1 on
```

Closure Memory

Function ko yaad rehna chahiye:

```
"Last value kya thi?"
```

Normal function yaad nahi rakhta.

Closure rakhta hai.

Solution Logic

Initial:

```
1 ["on", "off"]
```

First call

```
1 shift()
```

Gets:

```
1 on
```

Remaining:

```
1 ["off"]
```

Second call

Push previous.

Shift next.

Result:

```
1 off
```

Third call

Again rotate.

Result:

```
1 on
```

Essentially:

```
1 on → off → on → off → ...
```

Ya

```
1 slow
2 ↓
3 medium
4 ↓
5 fast
6 ↓
7 slow
```

Circular rotation.

Closure remembers current state.

4. Closure (Part 3)

Ye appendix ka sabse important exercise hai.

Goal:

Calculator create karna.

```
1 var calc = calculator();
```

Har keypress:

```
1 calc("4");
2 calc("+");
3 calc("3");
4 calc("=");
```

Output:

```
1 4+3=7
```

Main Closure Variables

```
1 var currentTotal = 0;  
2 var currentVal = "";  
3 var currentOper = "=";
```

Ye state hai.

Calculator ko remember rehna chahiye:

Current Number

```
1 73
```

User enter kar raha hai.

Store in:

```
1 currentVal
```

Current Operator

```
1 +
```

Store in:

```
1 currentOper
```

Current Total

```
1 75
```

Store in:

```
1 currentTotal
```

Example Flow

User:

```
1 4  
2 +  
3 73
```

4 =

After entering:

1 4

State:

1 currentVal = "4"

After:

1 +

State:

```
1 currentTotal = 4
2 currentOper = "+"
```

After:

1 73

State:

1 currentVal = "73"

After:

1 =

Execute:

1 4 + 73

Store:

1 currentTotal = 77

Closure ki wajah se ye saari state survive karti rehti hai.

Agar closure nahi hota to har call me values reset ho jaati.

Key Lesson

Calculator actual math nahi sikha raha.

Calculator excuse hai.

Real lesson:

Closure functions ko memory deta hai.

Yahi line author repeatedly emphasize kar raha hai.

5. Modules

Ab author calculator ko closure se module me convert karta hai.

Previous API

```
1  calc("4");
2  calc("+");
3  calc("3");
4  calc("=");
```

Single function.

New Module API

```
1  calc.number("4");
2  calc.plus();
3  calc.number("3");
4  calc.eq();
```

Methods alag hain.

Public API

```
1  {
2    number,
3    plus,
4    minus,
5    mult,
6    div,
7    eq
8  }
```

Ye object return ho raha hai.

Why Module?

Module bhi closure use karta hai.

Difference:

Closure version:

```
1 calc("+")
```

Single entry point.

Module version:

```
1 calc.plus()
```

Named entry points.

Zyada descriptive.

Pros of Module

Readability

```
1 calc.plus()
```

clear hai.

Better API Design

Methods self-documenting hain.

Easier Maintenance

Har operation alag function.

Extensibility

New method add kar sakte ho.

```
1 calc.sqrt()
```

```
1 calc.percent()
```

etc.

Cons of Module

Thoda zyada code.

```
1 calc.plus()
```

vs

```
1 calc("+")
```

Simple cases me extra boilerplate lag sakta hai.

Biggest Takeaway of Entire Appendix

Ye poora appendix essentially 4 cheeze reinforce karta hai:

1. Scope

Variable kahan visible hai?

2. Scope Chain

Variable lookup kaise hota hai?

3. Closure

Function outer variables ko remember kaise karta hai?

4. Module Pattern

Closure ko use karke private state + public API kaise banate hain?

Agar tum Buckets of Marbles, caching, toggle, calculator aur module exercises ko khud bina solution dekhe solve kar sakte ho, to Scope & Closures book ke most important concepts tumhe practically samajh aa gaye hain. Book 2 ka core goal bhi exactly yehi tha.